1	Canvas	# the (canvas) element defines a bitmapped are in an HTML page # the Canvas-API allows JavaScript to draw graphics on the canvas # the Canvas-API can draw – - shapes | lines | curves | boxes | text | images | colors | rotations - transparencies | and other pixel manipulations # (canvas) can access with the HTML DOM method (getElementById()) - to draw in the canvas need to create a 2D context object using (getContext()) # NB: the (canvas) element itself has no drawing abilities - must have to use JS to draw any graphics - (getContext()) method returns an object with tools (methods) for drawing # drawing on the canvas - after you have created a 2D context, you can draw in the canvas - (fillRect()) draws a back rectangle with a top-left corner at position 20, 20 - the rectangle is 150 pixel wide and 100 pixels high # using colors - the (fillStyle) property sets the fill color of the drawing object # can also create a new (canvas) with the (document.createElement()) method - and add the element to an existing HTML page # paths – the common way to draw on the canvas is to – - Begin a path – beginPath() - move to a point – moveTo() - draw in the path – lineTo() - draw the path – stroke() # complete canvas API Reference – - this reference covers all properties and Methods of the (getContext()) object, - used to draw (text | lines | boxes | circles | pictures | and more) on canvas # standard properties and Events - the canvas object also supports the standard properties and events
2	Audio / video	The HTML5 DOM has methods, properties, and events for the (audio | video) elements
3	Character sets	The default character set in HTML5 is UTF-8
4	Doctypes	# all HTML documents must start with a (!DOCTYPE) declaration # the declaration is not an HTML tag - it is an information to the browser about what document type to expect - the HTML5, the (!DOCTYPE) declaration is simple – (!DOCTYPE html) # in older document (HTML4 or XHTML), the declaration is more complicated because the declaration must refer to a DTD (Document Type Definition) - !DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.ort/TR/html14/loose.dtd" - !DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN" "http://www.w3.org/TR/xhtml11/DTD/xhtml111.dtd"
5	URL Encode	# web browsers request pages from web servers by using a URL - The URL is the address of a web page, like (https://www.w3schools.com)

		# URL Encoding (Percent Encoding) - URL encoding converts characters into a format that can be transmitted over the internet - URLs can only be sent over the internet using the ASCII character-set - since URLs often contain characters outside the ASCII set, the URL has to be converted into a valid ASCII format - URL encoding replaces unsafe ASCII characters with a (%) followed by two hexadecimal digits - URLs cannot contain spaces. URL encoding normally replaces a space with a plus (+) sign or with (%20) # URL encoding functions - in javascript, php and asp there are functions that can be used to URL encode a string - PHP has the rawurlencode() function, and ASP has the Server.URLEncode() function # ASCII encoding reference
6	Language codes	# should always include the (lang) attribute inside the (html) tag, - to declare the language of the web page - this is meant to assist search engines and browsers # ISO 639-1 language codes defines abbreviations for languages
7	Country codes	# ISO country codes - in HTML, country codes can be used as an additional to the language code in the (lang) attribute - the first two characters of a language code defines the language of the web page - the last two characters define the country # ISO country code
8	HTTP messages	# HTML error messages - when a browser requests a service from a web server, an error might occur, - and the server might return an error code like (404 Not found) - it is common to name these errors HTML error messages - but these messages are something called HTTP status messages. - in fact, the server always returns a message for every request. - the most common message is (200 ok) - below is a list of HTTP status messages that might be returned -
9	HTTP methods	# what is HTTP - the Hypertext Transfer Protocol (HTTP) is designed to enable communications between clients and servers - HTTP works as a request-response protocol between a client and server - the response contains status information about the request and may also contain the requested content # HTTP methods – - GET | POST | PUT | HEAD | DELETE | PATCH | OPTIONS | CONNECT | TRACE - the two most common HTTP methods are (GET and POST) # the GET method - GET is used to request data from a specified resource - NB: that the query string (name/value pairs) is sent in the URL of a GET request - /test/demo_form.php?name1=value1&name2=value2

		- some notes on GET requests - GET requests can be cached - GET requests remain in the browser history - GET requests can be bookmarked - GET requests should never be used when dealing with sensitive data - GET requests have length restrictions - GET request are only used to request data (not modify) # the POST method - POST is used to send data to a server to create/update a resource - the data sent to the server with POST is stored in the request body of the HTTP request – - POST /test /demo_from.php HTTP/1.1 - Host: w3schools.com -name1=value1&name2=value2 - some notes on POST requests - POST requests are never cached - POST request do not remain in the browser history - POST request cannot be bookmarked - POST requests have no restrictions on data length # compare GET vs. POST - BACK button/Reload - GET: harmless - POST: data will be re-submitted (the browser should alert the user that the data are about to be resubmitted) - bookmarked - GET: can be bookmarked - PSOT: cannot be bookmarked - cached - GET: can be cached - POST: not cached - encoding type - GET: application/x-www-form-urlencoded - POST: parameters are not saved in browser history - restrictions on data length - GET: yes, when sending data, the GET method adds the data to the URL, and the length of a URL is limited (maximum URL length is 2048 characters) - POST: no restrictions, binary data is also allowed - security - GET: GET is less secure compared to POST because data sent is part of the URL - PSOT: is a little safer than GET because the parameters are not stored in browser history or in web server logs - visibility - GET: data is visible to everyone in the URL - POST: data is not displayed in the URL

		# the PUT method - PUT is used to send data to a server to create/update a resource - the difference between POST and PUT is that PUT requests are idempotent. - that is, calling the same PUT request multiple times will always produce same result - in contrast, calling a POST request repeatedly have side effects of creating the same resource multiple times # the HEAD method - HEAD is almost identical to GET, but without the response body - in other words, if GET /users returns a list of users, then HEAD /users will make the same request but will not return the list of users - a HEAD request is useful for checking what a GET request will return before actually making a GET request – a HEAD request can read the content-length header to check the size of the file, without actually downloading the file # the DELETE method - the DELETE method deletes the specified resource # the PATCH method - the PATCH method is used to apply partial modifications to a resource # the OPTIONS method - the OPTIONS method describes the communication options for the target resource # the CONNECT method - the CONNECT method is used to start a two-way communication (a tunnel) with the requested resource # the TRACE method - the TRACE method is used to perform a message loop-back test that tests the path for the target resource (useful for debugging purposes)
10	HTML basic	# HTML is standard markup language for web pages which can create you web site # HTML References – - Elements | Browser Support | Attributes | Global Attributes | Event Attributes - color names | canvas | audio/video DOM | character sets | URL Encoding - language codes | country codes | HTTP messages | HTTP methods # HTML using Notepad - web pages can be created and modified by using professional HTML editors - learning HTML, recommend a simple text editor like (notepad) - after write some code of HTML in (notepad) save it on computer - conventionally, the first page name would be (index) - can use either (.htm | .html) as file extension, there is no difference - and set the encoding to (UTF-8), which is the preferred encoding for HTML file - after save the page, open the page in your favorite browser # to view HTML source

		- click (CTRL + U) in HTML page or (right-click) on the page and select (view source) - this will open a new tab containing the HTML source code of the page # to inspect an HTML element - (right-click) on an element (or a blank area), and choose (Inspect) - can also edit the HTML or CSS on-the-fly in the elements or styles panel that open # HTML documents structure - all HTML documents must start with a document type declaration - <! DOCTYPE html> - the HTML document itself begins with (html) and ends with (/html) - the visible part of the HTML document is between (body) and (/body) # the <! DOCTYPE > declaration - the <! DOCTYPE > declaration represents the document type, and helps browsers - to display web pages correctly - it must only appear once, at the top of the page (before any HTML tags) - the (DOCTYPE) declaration is not case-sensitive # the (HEAD | BODY) content - some head content like - title (title) | meta (meta) | base (base) | link (link) - some body content like - heading (h1 – h6) | paragraph (p) | link (a) | image (img) # elements - an hTML element is defined by a start tag, some content, and an end tag - the HTML element is everything from the start tag to the end tag - < tagname > content < /tagname > - NB: some HTML elements have no content (like the < br > element) - these elements are called empty elements - empty elements do not have an end tag - elements can be nested (this means that elements can contain other elements) - all HTML documents consist of nested HTML elements - NB: some HTML elements will display correctly without (end tag) but - should not skip the (end tag) may unexpected results or errors may occur - HTML tags are not case sensitive - the HTML standard does not require lowercase tags, but W3C recommends - lowercase in HTML, and demands lowercase for stricter doc types like XHTML # attributes - HTML attributes provide additional information about HTML elements - all HTML elements can have attributes - attributes provide additional information about elements - attributes are always specified in the (start tag) - attributes usually come in (name/value) pairs - the HTML standard does not require lowercase attribute names - however, W3C recommends lowercase attributes in HTML, - and demands lowercase attributes for stricter document types in XHTML - the HTML standard does not require quotes around attribute value - W3C recommends quotes in HTML,

		- and demands quotes for stricter document types like XHMTL - double quotes around attribute values are the most common in HTML, - but single quotes can also be used - in some situations, when the attributes value itself contains double quotes - it si necessary to use single quotes # comment - HTML comments are not displayed in the browser, but they can help document - your HTML source code - can add comments to HTML source by using the syntax - <!-- comment lines --> - NB: notice that there is an exclamation point (!) in the start tag, but not in end tag - comments can be used to hide content, - this can be helpful if you hide content temporarily - you can also hide more than one line - comments are also great for debugging HTML, because you can comment out - HTML lines of code, one at a time, to search for errors - comments can be used to hide parts in the middle of the HTML code # block and inline - every HTML element has a default display value, depending on what type of element it is - the two most common display values are (block and inline) # block-level element - a (block-level element always starts on a new line, and the browsers automatically add some space (a margin) before and after the element - a (block-level) element always takes up the full width available - (stretches out to the left and right as far as it can) - two commonly used block elements are (p | div), here (div) defines division or section in an HTML document - here are the (block-level) elements in HTML - address | article | aside | blockqute | canvas | dd | div | dl | dt | fieldset - figcaption | figure | footer | form | h1 – h6 | header | hr | li | main | nav - noscript | ol | p | pre | section | table | tfoot | ul | video # inline element - An (inline) element does not start on a new line - an (inline) element only takes up as much width as necessary - NB: an (inline) element cannot contain a (block-level) element - here are the (inline) elements in HTML - a | abbr | acronym | b | bdo | big | br | button | cite | code | dfn | em | I | img - input | kbd | label | map | object | output | q | samp | script | select | small - span | strong | sub | sup | textarea | time | tt | var # classes | div - the HTML (class) attribute is used to specify a class for an HTML element - multiple HTML elements can share the same (class) - using the (class) attribute -

		- the (class) attribute is often used to point to a class name in a style sheet - also used by JS to access and manipulate elements with specific class name - NB: the (class) attribute can be used on any HTML element - the (class) name is case-sensitive - NB: in CSS, to declare a class, used the period (.) character followed by class name, - then, define the properties within curly braces {} - HTML elements can belong to (more than one) class - to define (multiple) classes, separate the (class names) with a space, - the element will be styled according to all the classes specified - different HTML elements can point to the same class name - class name can also be used by JS to perform certain tasks for specific elements - JS can access elements with a specific class name with the (getElementsByClassName()) method - The HTML (id) attribute is used to specify a unique id for an HTML element - you cannot have more than one element with the same (id) in an HTML document - using the (id) attribute – - the (id) attribute specifies a unique id for an HTML element - the value of the (id) attribute must be unique within the HTML document - the (id) attribute is used to point to a specific style declaration in a style sheet - it is also used by JS to access and manipulate the element with the specific id - the syntax for (id) is – write a hash (#) character followed by an id-name - then, define the properties witting curly braces {} - NB: the (id) name is case-sensitive - NB: the (id) name must contain at least one character, cannot start with a number, - and must not contain whitespaces (spaces, tabs, etc) - (id) can also be used by JS to perform some tasks for that specific element - JS can access an element with a specific id with the (getElementById()) method - HTML (bookmarks) are used to allow readers to jump to specific parts of webpage - bookmarks can be useful if your page is very long - to use a bookmark, you must first create it, and then add a link to it - then , when the link is clicked, page will scroll to the location with the bookmark - difference between class and id - - a (class) name can be used by multiple HTML elements, while an (id) name must - only be used by one HTML element within the page # layout | responsive - websites often display content in multiple columns (like magazine or newspaper) - HTML layout elements – - HTML has several semantic elements that define the different parts of a web page - header | nav | section | article | aside | footer | details | summary - HTML layout techniques – - there are four different techniques to crate multicolumn layouts. - each technique has its pros and cons – - CSS framework | CSS float property | CSS flexbox | CSS grid - CSS frameworks –

		- if you want to create your layout fast, you can use a CSS framework, like – - W3.CSS | bootstrap - CSS Float layout – - it is common to do entire web layouts using the CSS (float) property - (float) is easy to learn – you just need to remember how the (float | clear) work - disadvantages: floating elements are tied to the document flow, which may harm - the flexibility - CSS Flexbox layout – - use of (flexbox) ensures that elements behave predictably when the page layout must accommodate different screen sizes and different display devices - CSS Grid layout – - the CSS (grid) layout module offers a (grid-based) layout system, with rows and columns, making it easier to design web pages without having to use floats and positioning - HTM responsive web design – - responsive web design is about creating web pages that look good on all devices - a responsive web design will automatically adjust for different screen sizes and viewports - what is responsive web design – - responsive web design is about using HTML and CSS to automatically - resize | hide | shrink | enlarge – a website to make it look good on all devices - like (desktops | tables | phones) - setting the viewport – - to create a responsive website, add the following (meta) tag to all you web pages - < meta name="viewport" content="width=device-width, initial-scale=1.0" > - this will set the viewport of your page, which will give the browser instructions on - how to control the page's dimensions and scaling - responsive image – - responsive images are images that scale nicely to fit any browser size - if the CSS (width) is set to 100%, image will be responsive and scale up and down - if the CSS (max-width) is set to 100%, image will scale down if it has to, but - never scale up to be larger than its original size - show different images depending on browser width - the HTML (picture) element allows you to define different images for different - browser window sizes - responsive text size – - the text size can be set with a (vw unit), which means the (viewport with) - that way the text size will follow the size of the browser window - NB: viewport is the browser window size. (1vw = 1%) of viewport width. - NB: if the viewport is (50cm) wide, (1vw is 0.5cm) - media queries – - in addition to resize text and images, it is also common to use media queries in responsive web page - with media queries you can define completely different styles for different browser sizes - responsive web design (framework) –

		- all popular CSS frameworks offer responsive design - W3.CSS – is a modern CSS framework with support for desktop | tablet | mobile - it is smaller and faster than similar CSS frameworks - it is designed to be independent of (jQuery) or any other JS library - another popular CSS framework is bootstrap # semantics - a semantic element clearly describes its meaning to both the browser and the developer - (non-semantic) element, like (div | span), tells nothing about its content - (semantic) element, like (form | table | article), clearly defines its content - many web sites contain HTML code, like - < div id="nav" > | < div class="header" > | < div id="footer" > - to indicate navigation, header, and footer - in HTML thre are some semantic elements that can be used to define different parts of a web page – - article | aside | details | figcaption | figure | footer | header | main | mark | nav - section | summary | time - why semantic elements – according to the W3C, a semantic web allows data to be shared and reused across applications, enterprises, and communities - HTML (section) element – - the (section) element defines a section in a document - according to W3C's HTML documentation – a section is a thematic grouping of content, typically with heading, (chapters | introduction | news items | contact) - a web page could normally be split into sections for information, content, etc - HTML (article) element – - the (article) element specifies independent, self-contained content - an article should make sense on its own, and it should be possible to distribute it independently form the rest of the web site, (forum posts | blog posts | user comments | product cards | newspaper articles) - nesting (article in section) – - the (article) element specifies independent, self-contained content - the (section) element defines section in a document - can we use the definitions to decide how to nest those elements, cannot - so, you will find HTML pages with (section) elements containing (article) elements, and (article) elements containing (section) elements - HTML (header) element – - the (header) element represents a container for introductory content or a set of navigational links - a (header) element typically contains - one or more heading elements (h1 – h6) | logo or icon | authorship information - NB: you can have several (header) elements in one HTML document, however (header) cannot be placed within a (footer | address | or another header) element - HTML (footer) element – - the (footer) element defines a footer for a document or section - a (footer) element typically contains - (authorship | copyright | contact | sitemap | back to top links | related docs) - you can have several (footer) elements in one document

		- HTML (nav) element – - the (nav) element defines a set of navigation links - NB: notice that NOT all links of a document should be inside a (nav) element - the (nav) element is intended only for major blocks of navigation links - browsers, such as screen readers for disabled users, can use this element to determine whether to omit the initial rendering of this content - HTML (aside) element – - the (aside) element defines some content aside from the content it is placed in (like a sidebar) - the (aside) content should be indirectly related to the surrounding content - HTML (figure | figcaption) element – - the (figure) tag specifies self-contained content, like (illustrations | diagrams | photos | code listings, etc) - the (figcaption) tag defines a caption for a (figure) element - the (figcaption) element can be placed as the first or as the last child of a (figure) element - the (img) element defines the actual image/illustration # styles and CSS - the HTML (style) attribute is used to add styles to an element, such as (color | font | size | and more) - setting the style of an HTML element, can be done with the (style) attribute - the HTML (style) attribute has the following syntax – - < tagName style="propertyName: value;" > - the (propertyName) is a CSS (property), the (value) is a CSS (value) - the css (background-color) property defines the background color of an element - the css (color) property defines text color on an element - the css (font-family) property defines the font of an element - the css (font-size) property defines the text size of an element - the css (text-align) property defines the horizontal text alignment of an element - CSS stands for Cascading Style Sheets - CSS saves a lot of work, it can control the layout of multiple web pages all at once - with CSS< you can control the (color | font | size of text | the spacing between elements, how elements are positioned and laid out, what background images or background colors are to be used, different display for different devices and screen sizes, and much more - NB: the word (cascading) means that a style applied to a parent element will also applied to all children element within the parent. So if you set the color of the body text to (blue), all headings, paragraphs, and other text elements within the body will also get the same color (unless you specify something else) - CSS ca be added to HTML documents in 3 ways – - inline – by using the (style) attribute inside HTML elements - internal – by using a (style) element in the (head) section or may in (body) - external – by using a (link) element to link to an external CSS file - the most common way to add CSS, is to keep the styles in external CSS files # javascript

		# style guide # colors | Favicon # entities # HTML vs XHTML # Forms # Graphics # media # APIs

	Drawing methods	
1	fillRect()	Draws a filled rectangle
2	strokeRect()	Draws a rectangle (with no fill)
3	clearRect()	Clears specified pixels within a rectangle
	Path methods	
4	beginPath()	Begins a new path or resets the current path
5	closePath()	Adds a line to the path from the current point to the start
6	isPointInPath()	Returns true if the specified point is in the current path
7	moveTo()	Moves the path to a point in the canvas (without drawing)
8	lineTo()	Adds a line to the path
9	Fill()	Fills the current path
10	Rect()	Adds a rectangle to the path
11	Stroke()	Draws the current path
12	BezierCurverTo()	Adds a cubic Bezier curve to the path (circles and curves)
13	Arc()	Adds an arc/curve (circle, or parts of a circle) to the path
14	arcTo()	Adds an arc/curve between two tangents to the path
15	quadraticCurveTo()	Adds a quadratic Bezier curve to the path
	Text	
16	Direction	Sets or returns the direction used to draw text
17	fillText()	Draws filled text on the canvas
18	Font	Sets or returns the font properties for text content
19	measureText()	Returns an object that contains the width of the specified text
20	strokeText()	Draws text on the canvas
21	textAlign	Sets or returns the alignment for text content
22	textBaseline	Sets or returns the text baseline used when drawing text
	Color, styles and shadows	
23	addColorStop()	Specifies the colors and stop positions in a gradient object
24	createLinearGradient()	Creates a linear gradient (to use on canvas content)
25	CreatePattern()	Repeats a specified element in the specified direction
26	createRadialGradient()	Creates a radial/circular gradient (to use on canvas content)
27	fillStyle	Sets or returns the color, gradient or pattern used to fill the drawing

28	lineCap	Sets or returns the style of the end caps for a line
29	lineJoin	Sets or returns the type of corner created, when two lines meet
30	linewidth	Sets or returns the current line width
31	miterLImit	Sets or returns the maximum miter length
32	shadowBlur	Sets or returns the blur level for shadows
33	shadowColor	Sets or returns the color to use for shadows
34	ShadowOffsetX	Sets or returns the horizontal distance of the shadow from the shape
35	ShadowOffsetY	Sets or returns the vertical distance of the shadow from the shape
36	strokeStyle	Sets or returns the color, gradient, or pattern used for strokes
	Transformations	
37	Scale()	Scales the current drawing bigger or smaller
38	Rotate()	Rotates the current drawing
39	Translate()	Remaps the (0,0) position on the canvas
40	Transform()	Replaces the current transformation matrix for the drawing
41	setTransform()	Resets current transform to identity matrix, then runs transform()
	Image drawing	
42	drawImage()	Draws an image, canvas, or video onto the canvas
	The ImageData Object / Pixel manipulation	
43	createImageData()	Creates a new, blank ImageData object
44	getImageData()	Returns an ImageData object that copies the pixel data for the specified rectangle on a canvas
45	ImageData.data	Returns an object that contains image data of a specified ImageData object
46	ImageData.height	Returns the height of an ImageData object
47	ImageData.width	Returns the width of an ImageData object
48	putImageData()	Puts the image data (from a specified ImageData object) back onto the canvas
	Compositing	
49	globalAlpha	Sets or returns the current alpha or transparency value of the drawing
50	globalCompositeOperation	Sets or returns how a new image are drawn onto an existing image
	Other methods	
51	Clip()	Clips a region of any shape and size from the original canvas
52	Save()	Saves the state of the current drawing context and all its attributes
53	Restore()	Restores the previously saved state and attributes
54	createEvent()	

55	getContext()	
56	toDataURL()	

	Audio \ video methods	
1	addTextTrank()	Adds a new text track to the audio/video
2	canPlayType()	Clicks if the browser can play the specified audio/video type
3	Load()	Re-loads the audio/video element
4	Play()	Starts playing the audio/video
5	Pause()	Pauses the currently playing audio/video
	Audio/video properties	
1	audioTracks	Returns an audioTrackList object representing available audio tracks
2	Autoplay	Sets or returns whether the audio/video should start playing as soon as it is loaded
3	Buffered	Returns a timeRanges object representing the buffered parts of the audio/video
4	Controller	Returns the mediaController object representing the current media controller to the audio/video
5	Controls	Sets or returns whether the audio/video should display controls (like play/pause etc)
6	currentSrc	Returns the URL of the current audio/video
7	CurrentTime	Sets or returns the current playback position in the audio/video (in seconds)
8	defaultMuted	Sets or returns whether the audio/video should be muted by default
9	defaultPlaybackRate	Sets or returns the default speed of the audio/video playback
10	Duration	Returns the length of the current audio/video (in second)
11	Ended	Returns whether the playback of the audio/video has ended or not
12	Error	Returns a mediaError object representing the error state of the audio/video
13	Loop	Sets or returns whether the audio/video should start over again when finished
14	mediaGroup	Sets or returns the group the audio/video belongs to (used to link multiple audio/video elements)
15	muted	Sets or returns whether the audio/video is muted or not
16	networkState	Returns the current network state of the audio/video
17	paused	Returns whether the audio/video is paused or not
18	playbackRate	Sets or returns the speed of the audio/video playback
19	played	Returns a TimeRanges object representing the played parts of the audio/video
20	preload	Sets or returns whether the audio/video should be loaded when the page loads
21	readyState	Returns the current ready state of the audio/video

22	seekable	Returns a TimeRanges object representing the seekable parts of the audio/video
23	seeking	Returns whether the user is currently seeking in the audio/video
24	src	Sets or returns the current source of the audio/video element
25	startDate	Returns a date object representing the current time offset
26	textTracks	Returns a TextTrackList object representing the available text racks
27	videoTracks	Deprecated. Do not use it
28	volume	Sets or return the volume of the audio/video
	Audio/video events	
29	abort	when the loading of an audio/video is aborted
20	canplay	when the browser can start playing the audio/video
21	canplaythrough	when the browser can play through the audio/video without stopping for buffering
22	durationchange	when the duration of the audio/video is changed
23	emptied	when the current playlist is empty
24	ended	when the current playlist is ended
25	error	when an error occurred during the loading of an audio/video
26	loadeddata	when the browser has loaded the current frame of the audio/video
27	loadedmetadata	When the browser has loaded meta data for the audio/video
28	loadstart	When the browser starts looking for the audio/video
29	pause	When the audio/video has been paused
30	play	When the audio/video has been started or is no longer paused
31	playing	When the audio/video is playing after having been paused or stopped for buffering
32	progress	When the browser is downloading the audio/video
33	ratechange	When the playing speed of the audio/video is changed
34	seeked	When the user is finished moving /skipping to a new position in the audio/video
35	seeking	When the user starts moving/skipping to a new position in the audio/video
36	stalled	When the browser is trying to get media data, but data in not available
37	suspend	When the browser is intentionally not getting media data
38	timeupdate	When the current playback position has changed
39	volumechange	When the volume has been changed
40	waiting	When the video stops because it needs to buffer the next frame

	1xx: information	
1	100 continue	The server has received the request headers, and the client should proceed to send the request body
2	101 switching protocols	The requester has asked the server to switch protocols
3	103 early hints	Used with the link header to allow the browser to start preloading resources while the server prepares a response
	2xx successful	

4	200 ok	The request is ok (this is the standard response for successful HTTP requests)
5	201 created	The request has been fulfilled, and a new resource is created
6	202 accepted	The request has been accepted for processing, but the processing has not been completed
7	203 non-authoritative information	The request has been successfully processed, but is returning information that may be from another source
8	204 no content	The request has been successfully processed, but is not returning any content
9	205 reset content	The request has been successfully processed, but is not returning any content, and requires that the requester rest the document view
10	206 partial content	The server is delivering only part of the resource due to a range header sent by the client
	3xx redirection	
11	300 multiple choices	A link list. The user can select a link and go to that location maximum five addresses
12	301 moved permanently	The requested page has moved to a new URL
13	302 found	The requested page has moved temporarily to a new URL
14	303 see other	The requested page has moved temporarily to a new URL
15	304 not modified	Indicates the requested page has not been modified since last requested
16	307 temporary redirect	The requested page has moved temporarily to a new URL
17	308 permanent redirect	The requested page has moved permanently to a new URL
	4xx client error	
18	400 bad request	The request cannot be fulfilled due to bad syntax
19	401 unauthorized	The request was a large request, but the server is refusing to respond to it. For use when authentication is possible but has failed or not yet been provided
20	402 payment required	Reserved for future use
21	403 forbidden	The request was a legal request, but the server is refusing to respond to it
22	404 not found	The requested page could not be found but may be available again in the future
23	405 method not allowed	A request was made of a page using a request method not supported by that page
24	406 not acceptable	The server can only generate a response that is not accepted by the client
25	407 proxy authentication required	The client must first authenticate itself with the proxy
26	408 request timeout	The server timed out waiting for the request
27	409 conflict	The request could not be completed because of a conflict in the request
28	410 gone	The requested page is no longer available

29	411 length required	The (content-length) in not defined. The server will not accept the request without it
30	412 precondition failed	The precondition given in the request evaluated to false by the server
31	413 request too large	The server will not accept the request, because the request entity is too large
32	414 request-URI too long	The server will not accept the request, because the URI is too long. Occurs when you convert a POST request to a GET request with a long query information
33	415 unsupported media type	The server will not accept the request, because the media type is not supported
34	416 range not satisfiable	The client has asked for a portion of the file, but the server cannot supply that portion
35	417 expectation failed	The server cannot meet the requirements of the expect request-header field
	5xx server error	
36	500 internal server error	A generic error message, given when no more specific messageis suitable
37	501 not implemented	The server either does not recognize the request method, or it lacks the ability to fulfill the request
38	502 bad gateway	The server was acting as a gateway or proxy and received an invalid response from the upstream server
39	503 service unavailable	The server is currently unavailable (overloaded or down)
40	504 gateway timeout	The server was acting as a gateway or proxy and did not receive a timely response from the upstream server
41	505 HTTP version not supported	The server does not support the HTTP protocol version used in the request
42	506 network authentication required	The client needs to authenticate to gain network access